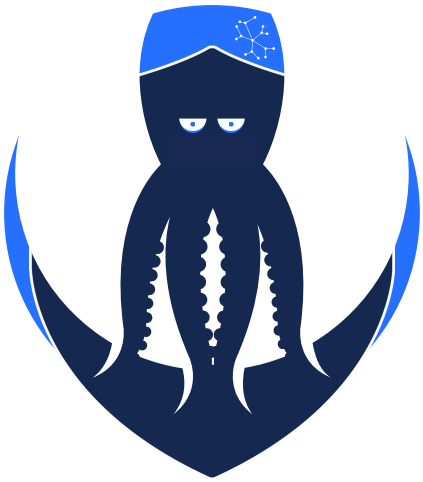


Math Master Initial Audit Report

Version 0.1

@aimanubayd

October 13, 2026

Math Master Audit Report

Aiman Ubayd

Oct 13, 2026

Math Master Audit Report

Prepared by: Aiman Ubayd
Lead Auditor: Aiman Ubayd
Assisting auditors: None

Table of contents

- Math Master Audit Report
- Table of contents
- About Aiman Ubayd
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
- Protocol Summary
  - Roles
- Executive Summary
  - Issues found
- Findings

- High
  - [H-1] `MathMasters::mulWadUp` increments `x` before calculating the rounded result
  - [H-2] `MathMasters::mulWadUp` overflow check can allow an overflowing product
  - [H-3] `MathMasters::sqrt` uses a threshold that differs from the reference implementation
- Medium
- Low
  - [L-1] `Solidity 0.8.3` cannot compile the library's custom errors
  - [L-2] `MathMasters__FullMulDivFailed()` may use the wrong selector
  - [L-3] `mulWad` and `mulWadUp` may revert with empty data
- Informational
  - [I-1] Custom-error data is written at the free-memory pointer

About Aiman Ubayd

Aiman Ubayd is a smart contract auditor and security researcher focused on how EVM and DeFi protocols fail, including protocol logic, economics, and trust boundaries. His review work combines manual analysis with invariant-driven fuzzing, formal verification, and AI-assisted security analysis.

- **X:** https://x.com/aimanubayd
- **GitHub:** https://github.com/aimanubayd
- **LinkedIn:** https://linkedin.com/in/aimanubayd
- **Contact:** aimanubayd@gmail.com

Disclaimer

This report records a time-bounded review of the Solidity code and audit material supplied by the author. A security review cannot guarantee that all vulnerabilities have been found, and it is not an endorsement of the project's business, team, or product. Findings and severity assessments depend on the reviewed source revision and the assumptions stated in this report. The exact review period, source revision, and scope were not supplied and must be completed before this report is treated as a final engagement record.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

Audit Details

The findings described in this document correspond the following commit hash:

```
675523145d60406a0adda17a900975a2499cba92
```

Scope

```
#-- src
|   #-- MathMasters.sol.sol
```

Protocol Summary

Math Master is presented as a Solidity mathematics library inspired by Solady and Solmate. The supplied materials discuss fixed-point multiplication and square-root routines, including `mulWadUp` and `sqrt`. No complete protocol architecture, deployment model, or downstream integrations were supplied, so this report does not infer how the library is used by an application or what funds may be at risk.

Roles

As a library, Math Master may have no privileged on-chain role; confirm this against the complete source before finalizing.

Executive Summary

Issues found

Severity	Number of issues found
High	3
Medium	0
Low	3
Info	1
Gas	0
Total	7

Findings

High

[H-1] `MathMasters::mulWadUp` increments `x` before calculating the rounded result

Description: `MathMasters::mulWadUp` increments `x` under certain conditions before calculating the product:

```
1  if iszero(sub(div(add(z, x), y), 1)) { x := add(x, 1) }
```

Because the named return value `z` starts at zero, the condition can be true when `x / y` rounds down to 1. The function then calculates the result using the modified `x`, in addition to applying its separate remainder-based rounding.

Impact: The function can return a value different from `ceil(x * y / WAD)`. Contracts relying on the library may therefore use incorrect arithmetic results. The practical impact depends on how callers use the result.

Proof of Concept: The supplied test uses `x = y = 3e18` and reports that mulWadUp returns `9000000000000000003` instead of the expected `9000000000000000000`:

```
function testMulWadUpCalculation() public {
    uint256 solution = (3e18 * 3e18) / 1e18;
    if (solution * 1e18 < 3e18 * 3e18) {
        solution += 1;
    }

    assertEq(MathMasters.mulWadUp(3e18, 3e18), solution);
}
```

Recommended Mitigation: Remove the increment and calculate the rounded result from the original inputs:

```
- if iszero(sub(div(add(z, x), y), 1)) { x := add(x, 1) }
  z := add(iszero(iszero(mod(mul(x, y), WAD))), div(mul(x, y), WAD))
```

Add tests for exact multiples, nonzero remainders, and boundary values.

[H-2] `MathMasters::mulWadUp` overflow check can allow an overflowing product

Description: The overflow check uses `or` to combine the division result and `x`:

```
if mul(y, gt(x, or(div(not(0), y), x))) {
    // revert
}
```

This changes the comparison threshold. For `x = 2` and `y = 2^255`, `type(uint256).max / y is 1`; `or(1, 2)` is 3. Therefore `gt(2, 3)` is false, and the function does not revert before multiplying.

Impact: An overflowing product can pass the check, causing the function to calculate from the wrapped product instead of reverting. Callers relying on this check may receive an incorrect result.

Proof of Concept: The supplied test constrains `x` to values from 2 to 9, chooses `y > type(uint256).max / x`, and expects a revert:

```
function testMulWadUpFuzzOverflow(uint256 x, uint256 y) public {
    vm.assume(x > 1 && x < 10);
    vm.assume(y > type(uint256).max / x);

    vm.expectRevert();
    MathMasters.mulWadUp(x, y);
}
```

Recommended Mitigation: Compare `x` directly with `type(uint256).max / y`:

```
- if mul(y, gt(x, or(div(not(0), y), x))) {
+ if mul(y, gt(x, div(not(0), y))) {
      // revert
  }
```

Add tests for zero inputs and values immediately around the overflow boundary.

[H-3] `MathMasters::sqrt` uses a threshold that differs from the reference implementation

Description: The supplied `sqrt` implementation compares `shr(r, x)` against `16777002 (0xffff2a)`, while the referenced Solady implementation uses `0xffffff (16777215)`. These are different numeric values. The issue is the constant mismatch, not whether the constants are written in decimal or hexadecimal.

Impact: The threshold affects the initial estimate used by the square-root routine and may cause incorrect results for some inputs. The supplied material reports a Halmos counterexample but does not include the resulting outputs, so the mismatch should be confirmed against the exact source revision.

Proof of Concept: The submitted PoC compares the core MathMasters and Solady routines and reports that Halmos found a counterexample. It supplies this input for a Foundry assertion:

```
uint256 x =
    105311293498665291426722909308999732236070323463302251608708546560;

uint256 mathMastersOutput = Base_Test.coreMathMasterSqrt(x);
uint256 soladyOutput = Base_Test.coreSoladySqrt(x);

assertEq(mathMastersOutput, soladyOutput);
```

Recommended Mitigation: Use the reference threshold and test around the relevant input boundaries:

```
- r := or(r, shl(4, lt(16777002, shr(r, x))))
+ r := or(r, shl(4, lt(0xffffff, shr(r, x))))
```

Medium

No medium-severity findings.

Low

[L-1] `Solidity 0.8.3` cannot compile the library's custom errors

Description: The library uses custom errors, which Solidity `0.8.3` does not support. The supplied PoC imports the library into a contract pinned to `pragma solidity 0.8.3` and reports compilation errors at the custom-error declarations.

Impact: Contracts restricted to Solidity `0.8.3` cannot compile while importing this library. This is a compatibility issue rather than a runtime vulnerability.

Proof of Concept: The supplied example uses:

```
pragma solidity 0.8.3;

import {MathMasters} from "src/MathMasters.sol";

contract MathMastersExposed {
    using MathMasters for uint256;
}
```

The submission reports that `forge build` fails on custom-error declarations.

Recommended Mitigation: If Solidity `0.8.3` compatibility is not required, set the library pragma to a version that supports custom errors:

```
- pragma solidity ^0.8.3;
+ pragma solidity ^0.8.4;
```

[L-2] `MathMasters__FullMulDivFailed()` may use the wrong selector

Description: The selector for `MathMasters__FullMulDivFailed()` is `0x41672c55`, while the implementation uses `0xae47f702`.

Impact: Callers or tests that inspect revert data may fail to identify the intended error.

Proof of Concept:

Recommended Mitigation: Verify the selector against the exact custom-error declaration, use the matching selector in the revert path, and add a test that asserts the exact revert data.

[L-3] `mulWad` and `mulWadUp` may revert with empty data

Description: In the `mulWad` and `mulWadUp` snippets, the implementation stores a selector at memory offset `0x40` but reverts using bytes at offset `0x1c`:

```
mstore(0x40, 0xbac65e5b)
revert(0x1c, 0x04)
```

Also `0xbac65e5b` is not the selector for `MathMasters__MulWadFailed()` and gives `0xa56044f7` as the expected selector.

Impact: The functions may return empty or incorrect revert data instead of the intended custom error. This makes failures harder for callers, integrations, and tests to identify.

Proof of Concept: Assembly shows that the selector is stored at `0x40`, while the revert reads `0x1c`.

Recommended Mitigation: Store the correct selector where the revert reads it, in both functions:

```
- mstore(0x40, 0xbac65e5b)
+ mstore(0x00, 0xa56044f7)
  revert(0x1c, 0x04)
```

Add tests that assert the exact custom error for each function. ## Informational ### [I-1] Custom-error data is written at the free-memory pointer

Description: Custom-error data is written at memory offset 0x40, conventionally used for the free-memory pointer. It does not demonstrate that this causes a functional problem.

Impact:

Proof of Concept:

Recommended Mitigation: